Predictive Modeling using

Classification



Using machine learning algos

DS Week 7: Classification

Predictive Modeling with Machine Learning Algorithms

We are going from data analytics to data SCIENCE.

Instead of just seeing what our data shows, we will now use it to predict the future.

(Content is hidden under this rectangle. Start the slideshow to view.)

@19:54



Jargon

Let’s dive right into the ball pit with words you don’t know.



The two main types of data there are.

This “data” refers to a row, not a single cell.

Labeled Data

Used for Regression and Classification.

When your data has a variable that you are trying to predict.

Unlabeled Data

Used for clustering.

When your data does not have the variable that you are trying to predict.

Clustering v. Classification v. Regression

Equivalent*

	Labeled Data	Supervised	Output	Example Algorithm	Example Application
Clustering	❌	❌	Clusters	K-means	Group cosmic structures
Our Focus **Classification**	✅	✅	Classes (or Categories)	Logistic regression	Identify an animal species
Regression	✅*	✅	Continuous values	Linear regression	Forecast profits

Typology of Data

		Type	Use Case	Orderable	Example
Qualitative	**Nominal**	Categorical group (or quality)	Categorizing, classifying, or grouping	❌	{cat, dog, rat}
	Ordinal			✅	NY restaurant grade {A, B, C}
Quantitative	**Discrete**	Numeric amount (or quantity)	Counting	✅	Number of people
	Continuous		Measuring	✅	Temperature

Subdivisible into “Interval” and “Ratio”

Classification and Regression in a Nutshell

1. Get a boatload of clean, **labeled** data.
2. Select the **features** to use and a **target** to predict.
3. **Train** (**fit**) a **model**.
4. Use the model for **prediction** on some data it has not seen before / you don’t have the answer to.
5. Evaluate the predictions

Jargon Alert

Features are the INDEPENDENT VARIABLES

They are data that we use to predict the label

- Aka columns, information, data, “X”, independent variables.
- Independent variables (explain later why they ideally should not be related in any way // multicollinearity)

Jargon Alert

Features are the INDEPENDENT VARIABLES

They are data that we use to predict the label

- Aka columns, information, data, “X”, independent variables.
- Independent variables (explain later why they ideally should not be related in any way // multicollinearity)

Target is the DEPENDENT VARIABLE

It is thing(s) you are trying to predict using the features.

- Aka labels, classes, target_variable, ground truth, “y”,
- Dependent variable (as it depends on the independent variables)

A Bare-Minimum Cookie-cutter Template

```
######################################## Prep ########################################

from sklearn.linear_model import LogisticRegression # Import your model types and
import …                                            # anything else you need.

…
df = …                                              # Get your pandas DataFrame ready.

features = ['fare', 'age']                          # Select your feature columns and
target = 'survived'                                 # your target column.

X = df[features]                                    # Define your X (features DataFrame) and
y = df[target]                                      # your y (target Series).

model = LogisticRegression()                        # Instantiate your model.

model.fit(X, y)                                     # Feed your features and target into it.

###################################### Deploy/use ######################################

…
some_other_X = …[features]                          # Get some unseen data's features.

y_pred = model.predict(some_other_X)                # Predict the target from them.

print(y_pred)                                       # An example of what you can do with the
                                                    # predicted values.
```

```
['Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Lived' 'Died' 'Died' 'Died'
 'Lived' 'Died' 'Died' 'Lived' 'Died' 'Died' 'Lived' 'Died' 'Lived' 'Lived' 'Died' 'Lived'
 'Died' 'Died' 'Died' 'Died' 'Lived' 'Lived' 'Died' 'Died' 'Died' 'Lived' 'Died' 'Lived' …
```

1. Get a boatload of clean, labeled data.
2. Select the features to use and a target to predict.
3. Train (fit) a model.

4. Use the model for prediction on some other data.

🆗 but is it a good model?

```python
# Select your features and target variables
selected_features = ['fare', 'age']
target_feature = 'survived'

# Define your X and y
X = df[selected_features]
y = df[target_feature]

# Split your data into train and test.
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Initalize your model
model = LogisticRegression()

# Fit your model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate your model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score: %f" % accuracy)

f1 = f1_score(y_test, y_pred)
print("F1 Score: %f" % f1)
```

[11] ✓ 0.0s Python

```
Accuracy Score: 0.664804
F1 Score: 0.375000
```

Features aka IVs

	FEATURE 1	FEATURE 2	FEATURE 3	FEATURE 4	TARGET!!
	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.8	1.6	0.2	Iris-setosa
1	5.0	3.6	1.4	0.2	Iris-setosa
2	6.1	2.9	4.7	1.4	ris-versicolor
3	7.6	3.0	6.6	2.1	Iris-virginica
4	7.2	3.0	5.8	1.6	Iris-virginica
5	4.9	2.4	3.3	1.0	ris-versicolor
6	5.1	3.3	1.7	0.5	Iris-setosa
7	5.1	3.5	1.4	0.2	Iris-setosa
8	7.0	3.2	4.7	1.4	ris-versicolor
9	7.7	2.8	6.7	2.0	Iris-virginica

```
1
2  features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
3  target_variable = 'class'
4
5  X = df[features]
6  y = df[target_variable]
```

[69] ✓ 0.0s Python

	FEATURE 1	FEATURE 2	FEATURE 3	FEATURE 4	TARGET!!
	sepal_length	sepal_width	petal_length	petal_width	class
0	5.1	3.8	1.6	0.2	Iris-setosa
1	5.0	3.6	1.4	0.2	Iris-setosa
2	6.1	2.9	4.7	1.4	ris-versicolor
3	7.6	3.0	6.6	2.1	Iris-virginica
4	7.2	3.0	5.8	1.6	Iris-virginica
5	4.9	2.4	3.3	1.0	ris-versicolor
6	5.1	3.3	1.7	0.5	Iris-setosa
7	5.1	3.5	1.4	0.2	Iris-setosa
8	7.0	3.2	4.7	1.4	ris-versicolor
9	7.7	2.8	6.7	2.0	Iris-virginica

```
1
2  features = ['sepal_length', 'sepal_w
3  target_variable = 'class'
4
5  X = df[features]
6  y = df[target_variable]
```

[69] ✓ 0.0s

```
1
2  features = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
3  target_variable = 'class'
4
5  X = df[features]
6  y = df[target_variable]
7
8  logit = LogisticRegression()
9  logit.fit(X, y)
10
11 y_pred = logit.predict(X)
12 print(y_pred)
13
```

[5] ✓ 0.0s Python

```
['Iris-setosa' 'Iris-virginica' 'Iris-setosa' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-virginica' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-versicolor' 'Iris-setosa' 'Iris-virginica'
 'Iris-setosa' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa' 'Iris-versicolor'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor' 'Iris-virginica'
```

	FEATURE 1	FEATURE 2	FEATURE 3	FEATURE 4	TARGET
	sepal_length	sepal_width	petal_length	petal_width	cla
0	5.1	3.8	1.6	0.2	Iris-setosa
1	5.0	3.6	1.4	0.2	Iris-setosa
2	6.1	2.9	4.7	1.4	ris-versicolor
3	7.6	3.0	6.6	2.1	Iris-virginica
4	7.2	3.0	5.8	1.6	Iris-virginica
5	4.9	2.4	3.3	1.0	ris-versicolor
6	5.1	3.3	1.7	0.5	Iris-setosa
7	5.1	3.5	1.4	0.2	Iris-setosa
8	7.0	3.2	4.7	1.4	ris-versicolor
9	7.7	2.8	6.7	2.0	Iris-virginica

Classification in a Nutshell

Target predicts Features

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	relatives	not_alone	Deck	Title	Age_Class
Testing	😵	3	0	2	1	0	0	0	1	0	8	1	6
	🙂	1	1	5	1	0	3	1	1	0	3	3	5
	🙂	3	1	3	0	0	0	0	0	1	8	2	9
Training	🙂	1	1	5	1	0	3	0	1	0	3	3	5
	😵	3	0	5	0	0	1	0	0	1	8	1	15
	😵	3	0	4	0	0	1	2	0	1	8	1	12
	😵	1	0	6	0	0	3	0	0	1	5	1	6
	😵	3	0	0	3	1	2	0	4	0	8	4	0
	🙂	3	1	3	0	2	1	0	2	0	8	3	9
	🙂	2	1	1	1	0	2	1	1	0	8	3	2
	🙂	3	1	0	1	1	2	0	2	0	7	2	0

What might the features and target for the Iris data set be?

→ Col. A	Col. B	Col. C	Col. D	Col. E	Col. F
Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa
7	4.6	3.4	1.4	0.3	Iris-setosa
8	5	3.4	1.5	0.2	Iris-setosa
9	4.4	2.9	1.4	0.2	Iris-setosa
10	4.9	3.1	1.5	0.1	Iris-setosa
11	5.4	3.7	1.5	0.2	Iris-setosa
12	4.8	3.4	1.6	0.2	Iris-setosa
13	4.8	3	1.4	0.1	Iris-setosa
14	4.3	3	1.1	0.1	Iris-setosa
15	5.8	4	1.2	0.2	Iris-setosa
16	5.7	4.4	1.5	0.4	Iris-setosa
17	5.4	3.9	1.3	0.4	Iris-setosa
18	5.1	3.5	1.4	0.3	Iris-setosa
19	5.7	3.8	1.7	0.3	Iris-setosa

Continuous Features

Nominal Target

If we are trying to predict if someone died or survived on the titanic.

What are our feature columns?

- *Aka* **X**

What is our target_variable column.

- *Aka* **y**

	fare	sex	age	pclass	survived
0	14.0	male	59.0	2	Died
1	30.0	female	25.0	2	Lived
2	27.0	female	24.0	2	Lived
3	8.0	male	32.0	3	Lived
4	13.0	male	36.0	2	Died
5	7.0	male	17.0	3	Died
6	26.0	male	54.0	2	Died
7	67.0	male	29.0	1	Died
8	21.0	female	35.0	2	Lived
9	23.0	female	4.0	2	Lived

If we are trying to predict if someone died or survived on the titanic.

What are our feature columns?

- ***Aka X***

What is our target_variable column.

- ***Aka y***

	FEATURE 1	FEATURE 2	FEATURE 3	FEATURE 4	TARGET!!
	fare	sex	age	pclass	survived
0	14.0	male	59.0	2	Died
1	30.0	female	25.0	2	Lived
2	27.0	female	24.0	2	Lived
3	8.0	male	32.0	3	Lived
4	13.0	male	36.0	2	Died
5	7.0	male	17.0	3	Died
6	26.0	male	54.0	2	Died
7	67.0	male	29.0	1	Died
8	21.0	female	35.0	2	Lived
9	23.0	female	4.0	2	Lived

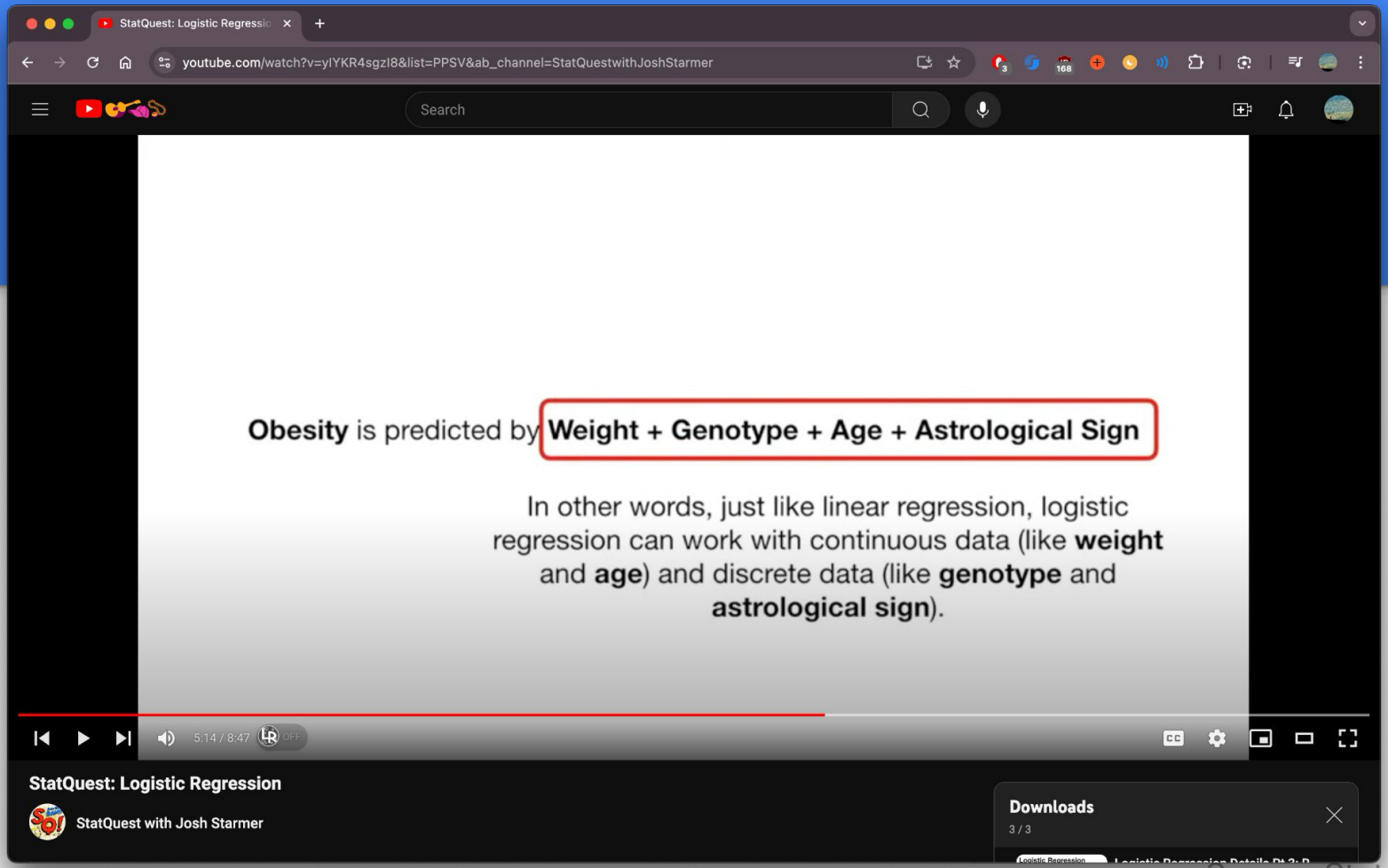




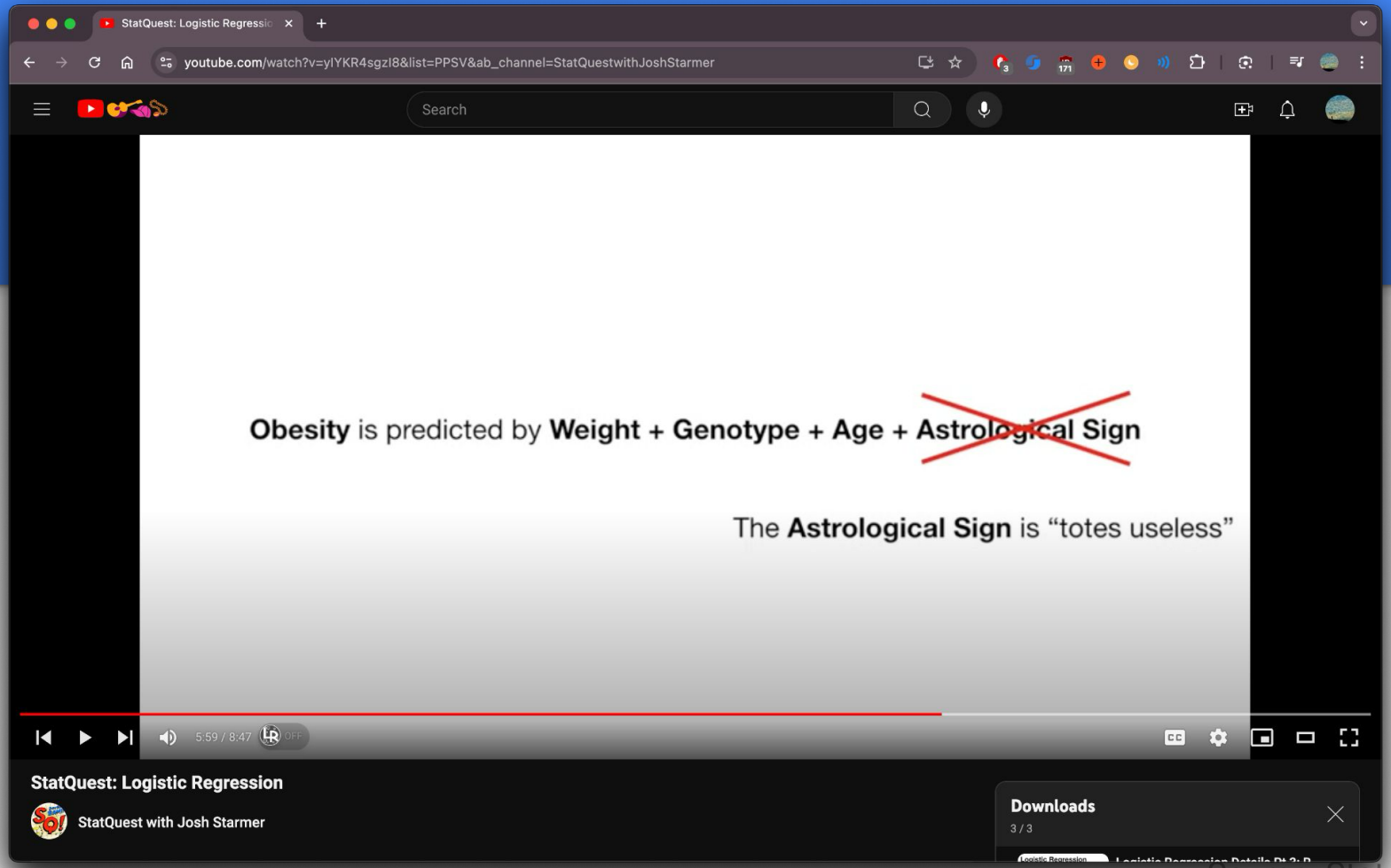




passengerid	pclass	name	sex	age	sibsp	parch	ticket	fare	cabin	embarked	survived
450	1	Peuchen, Major. Arthur Godfrey	male	52.0	0	0	113786	30.0	C104	S	Lived
713	1	Taylor, Mr. Elmer Zebley	male	48.0	1	0	19996	52.0	C126	S	Lived
454	1	Goldenberg, Mr. Samuel L	male	49.0	1	0	17453	89.0	C92	C	Lived
858	1	Daly, Mr. Peter Denis	male	51.0	0	0	113055	27.0	E17	S	Lived
631	1	Barkworth, Mr. Algernon Henry Wilson	male	80.0	0	0	27042	30.0	A23	S	Lived
264	1	Harrison, Mr. William	male	40.0	0	0	112059	0.0	B94	S	Died
718	2	Troutt, Miss. Edwina Celia "Winnie"	female	27.0	0	0	34218	10.0	E101	S	Lived
175	1	Smith, Mr. James Clinch	male	56.0	0	0	17764	31.0	A7	C	Died
807	1	Andrews, Mr. Thomas Jr	male	39.0	0	0	112050	0.0	A36	S	Died
588	1	Frolicher-Stehli, Mr. Maxmillian	male	60.0	1	1	13567	79.0	B41	C	Lived

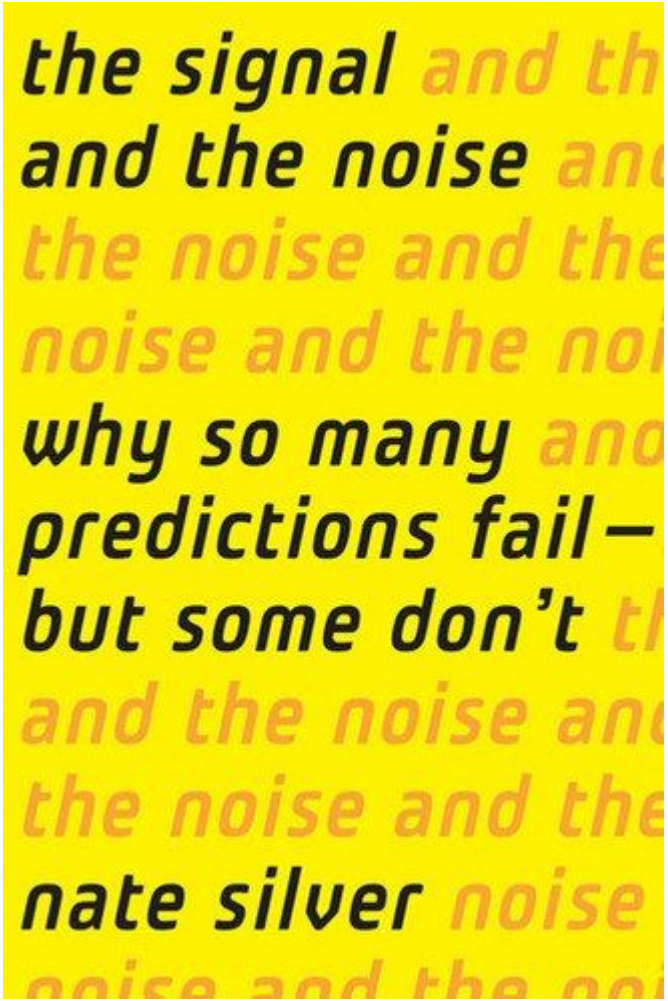


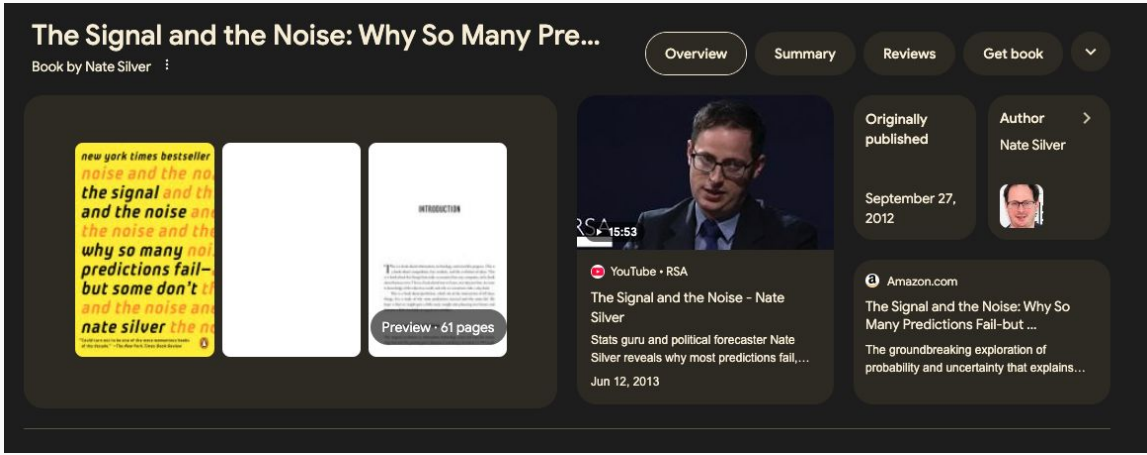
Source: StatsQuest



Source: StatsQuest

signals


the signal and the noise
why so many predictions fail—
but some don't
nate silver


The Signal and the Noise: Why So Many Pre...
Book by Nate Silver
Overview
Summary
Reviews
Get book
new york times bestseller
Preview · 61 pages
15:53
YouTube • RSA
The Signal and the Noise - Nate Silver
Stats guru and political forecaster Nate Silver reveals why most predictions fail,...
Jun 12, 2013
Originally published
September 27, 2012
Author
Nate Silver
Amazon.com
The Signal and the Noise: Why So Many Predictions Fail-but ...
The groundbreaking exploration of probability and uncertainty that explains...

signals

This pattern you will use in almost all supervised learning.

X = df[features]
y = df[target_varible]

model.fit(X, y)
model.predict(X)

```python
from sklearn.linear_model import LogisticRegression

# Select your features and target
features = ['fare', 'age']
target_variable = 'survived'

# Define your X and y
X = df[features]
y = df[target_variable]

# Create an empty model
logit = LogisticRegression()

# Do the math
logit.fit(X, y)

# Make predictions
y_pred = logit.predict(X)

# Print your predictions
print(y_pred)

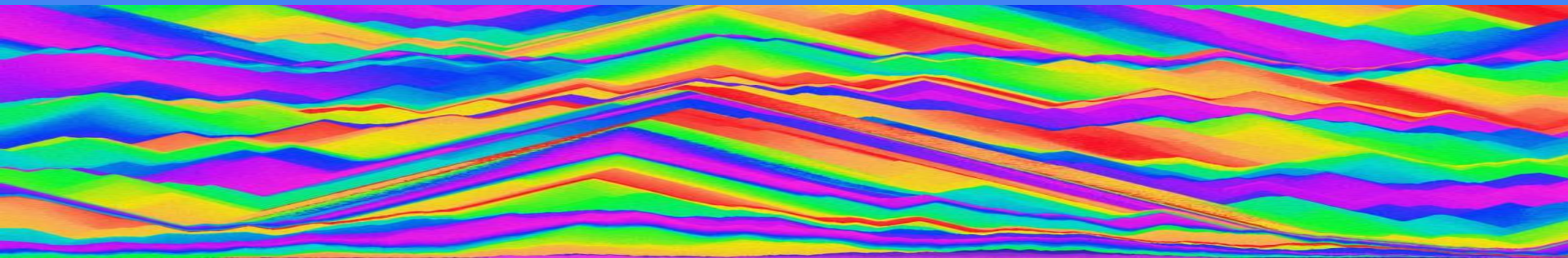
```

✓ 0.0s

```
['Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Died' 'Lived'
 'Died' 'Died' 'Died' 'Lived' 'Died' 'Died' 'Lived' 'Died' 'Died' 'Lived'
 'Died' 'Lived' 'Lived' 'Died' 'Lived' 'Died' 'Died' 'Died' 'Died' 'Lived'
 'Lived' 'Died' 'Died' 'Died' 'Lived' 'Died' 'Lived' 'Died' 'Died' 'Died'
```

Our first classification algorithm

Logit





~~Logit == Logistic Regression~~

Logit == Logistic
Classification

Logistic Regression is Classification

1. Logistic regression is classification.
2. Logistic regression is classification.
3. Logistic regression is classification.

This is scary (linear algebra ahh)

Taking the natural logarithm of the odds gives the log-odds or logit:

$$\log\left[\frac{p(x)}{1-p(x)}\right] = z$$

$$\log\left[\frac{p(x)}{1-p(x)}\right] = w \cdot X + b$$

$$\frac{p(x)}{1-p(x)} = e^{w \cdot X + b} \quad \cdots \text{Exponentiate both sides}$$

$$p(x) = e^{w \cdot X + b} \cdot (1 - p(x))$$

$$p(x) = e^{w \cdot X + b} - e^{w \cdot X + b} \cdot p(x))$$

$$p(x) + e^{w \cdot X + b} \cdot p(x)) = e^{w \cdot X + b}$$

$$p(x)(1 + e^{w \cdot X + b}) = e^{w \cdot X + b}$$

$$p(x) = \frac{e^{w \cdot X + b}}{1 + e^{w \cdot X + b}}$$

then the final logistic regression equation will be:

$$p(X; b, w) = \frac{e^{w \cdot X + b}}{1 + e^{w \cdot X + b}} = \frac{1}{1 + e^{-w \cdot X + b}}$$

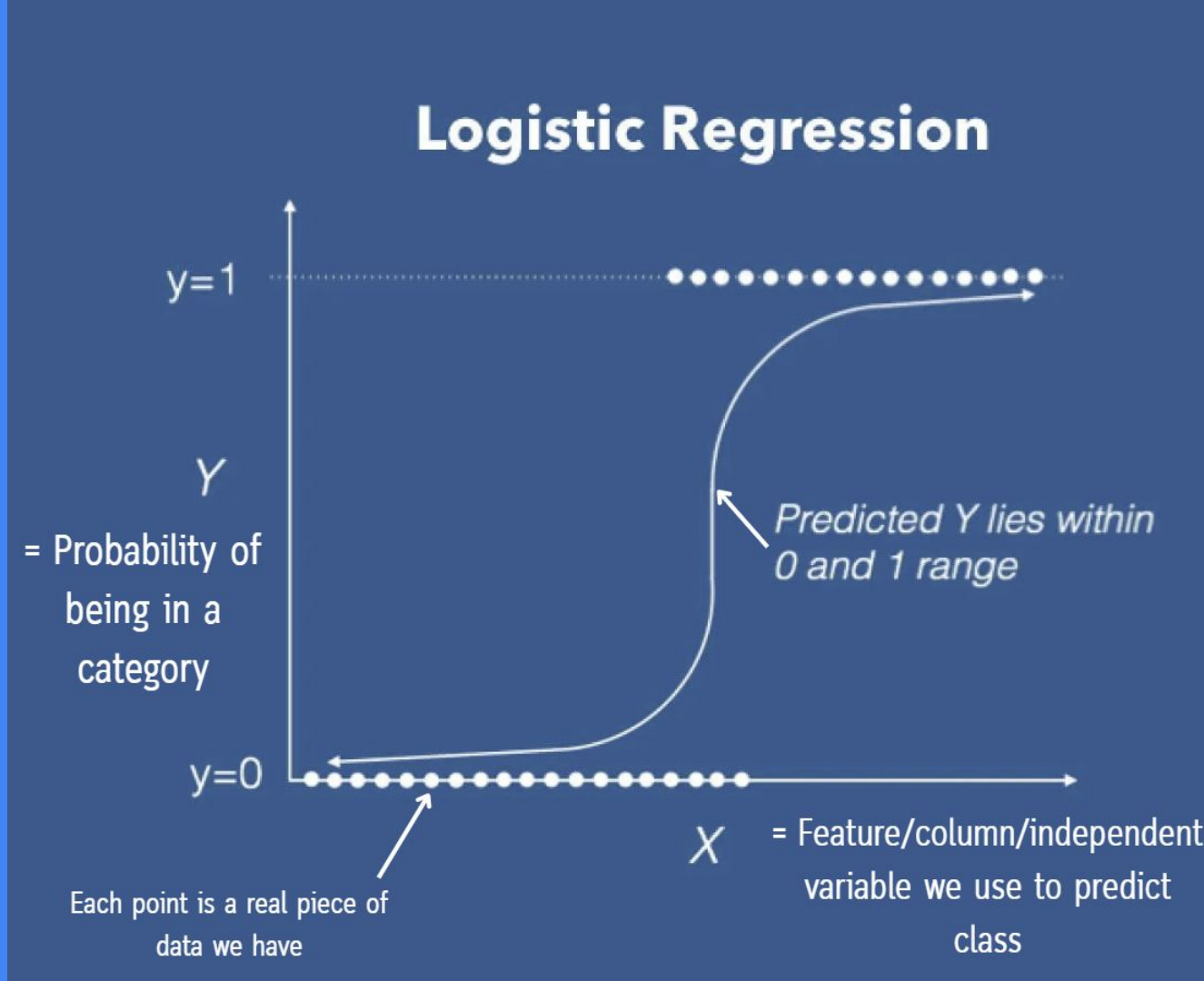
Logistic Regression

doesn't have to be scary

A good starting point into machine learning

Graph features:

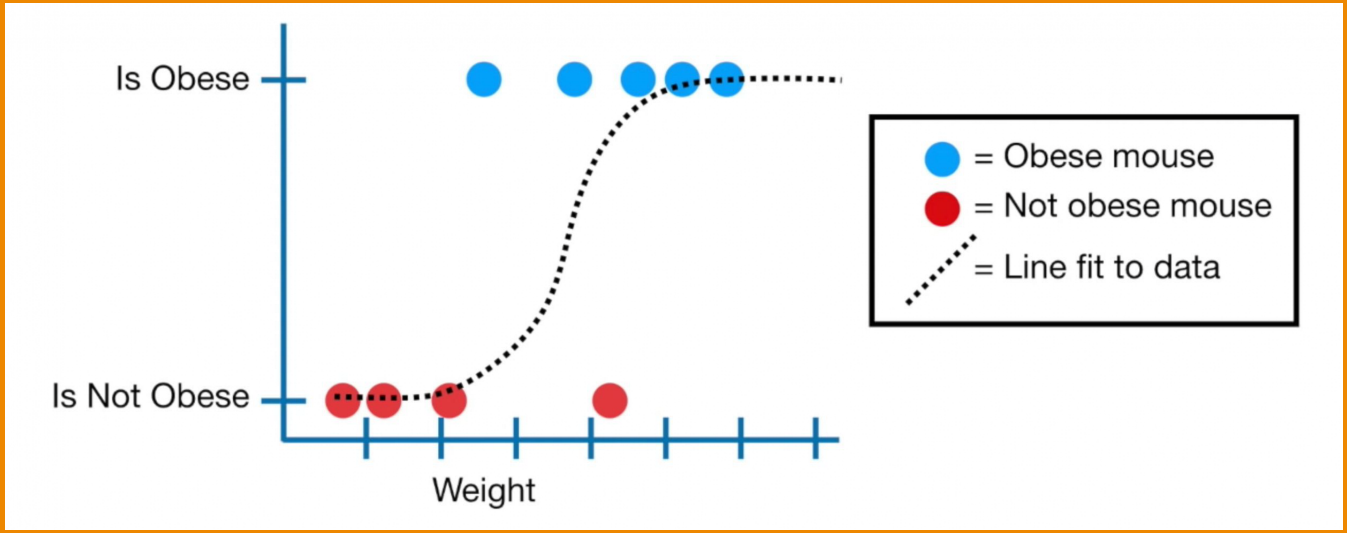
- Data points can only be 0 or 1
- Function predicts whether or not something is in the class based on the X variable







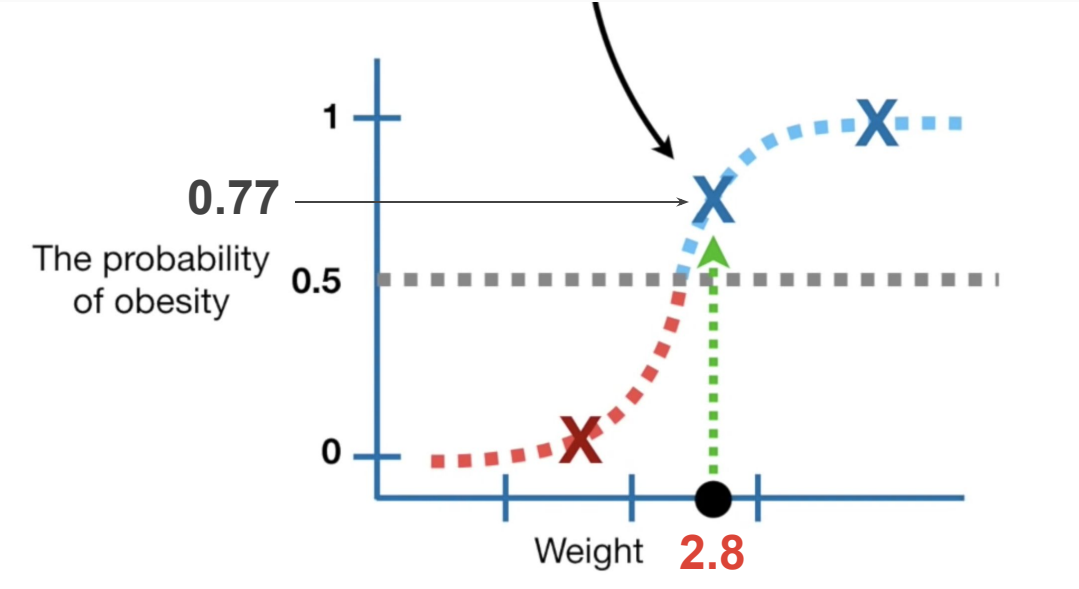
Data for this example



FEATURE	TARGET!!
WEIGHT	IS OBESE
2.1	YES
3.1	YES
5.5	YES
0.4	NO
1.1	NO
2.1	NO

Predicted Probability

This mouse that has a weight of 2.8 has a 77% probability of being obese.



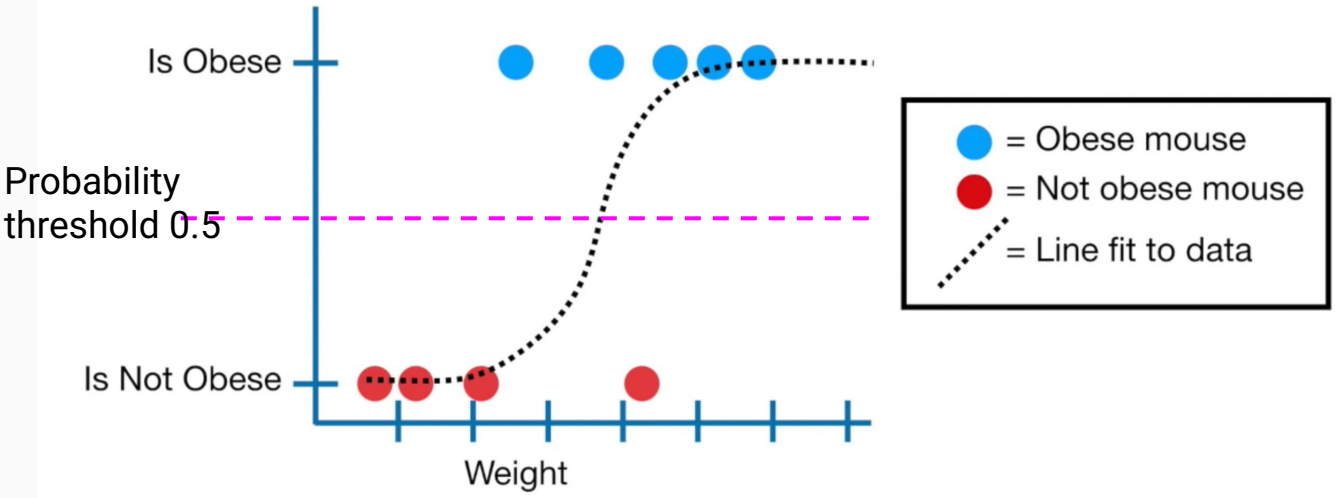
Source: StatsQuest

The Sigmoid Function
Aka how it separates your data.

The sigmoid function is used to separate the the data into two areas by the probability threshold.

The probability threshold is the cutoff for deciding what class something is in.

The x coordinate of the point where the Sigmoid function = threshold is how we separate the classes.



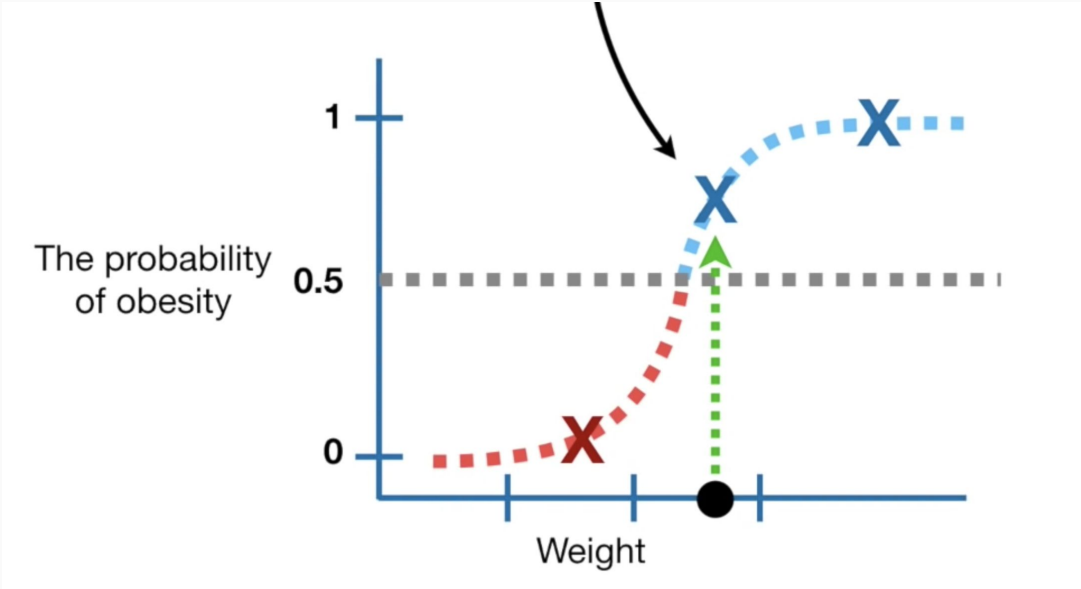
Source: StatsQuest

Classification threshold
Above 0.5 is obese below is not obese.

Any weight that falls above the 0.5 probability threshold (the blue) will be classified as obese.

Everything falling below the 0.5 probability threshold will be classified as not obese.

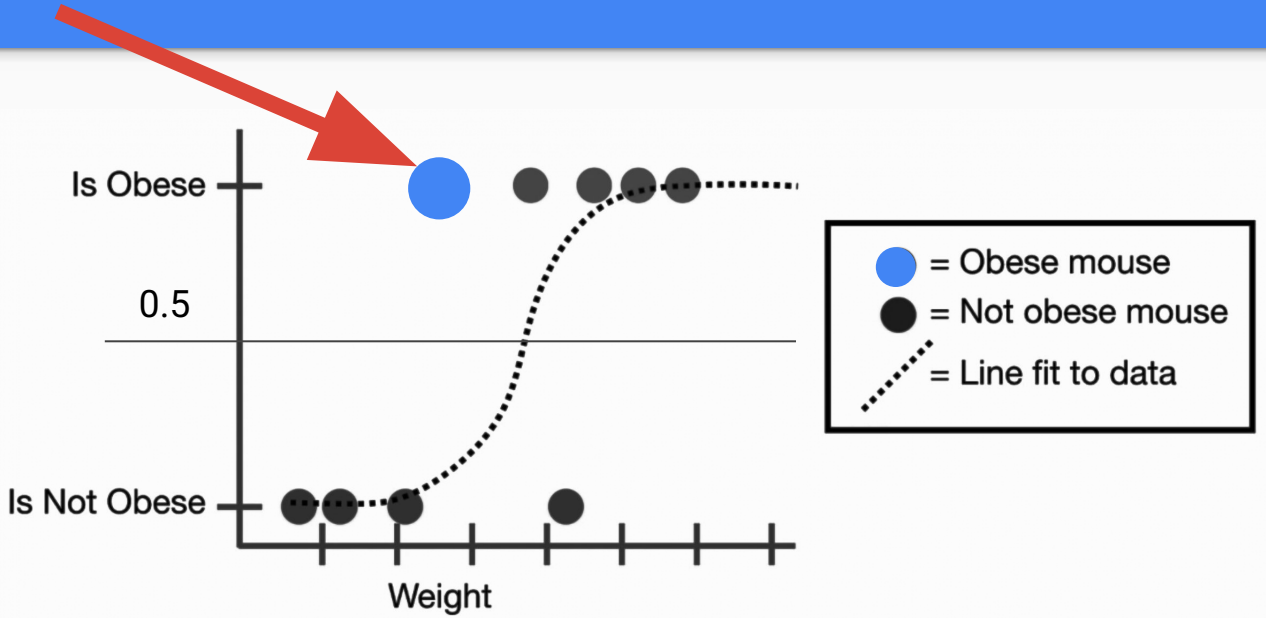
We can change this threshold. For this lecture, we are going to assume the 0.5 threshold.



Source: StatsQuest

Intuition check.
Why might this mouse be obese when it weighs so little?

What other FEATURES could we consider?

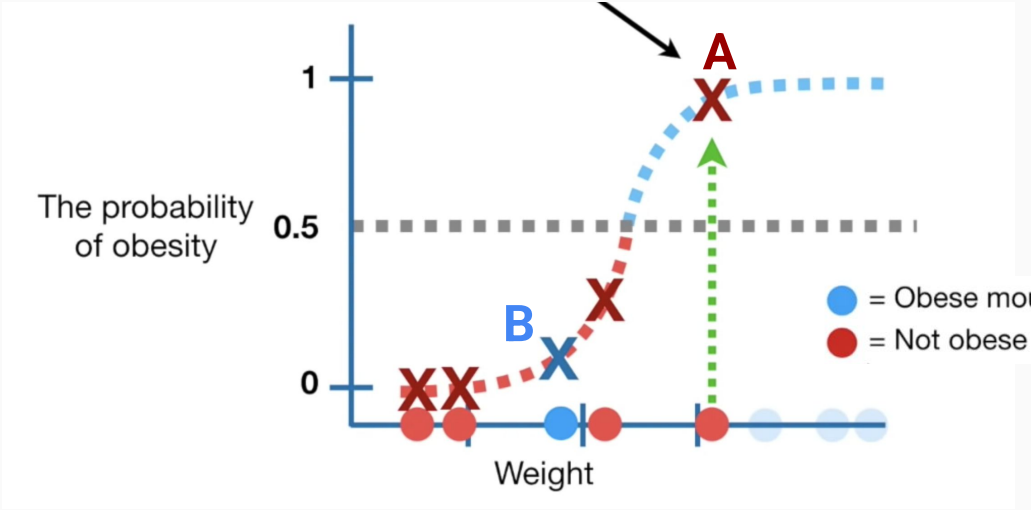


Source: StatsQuest

Classifiers can get things wrong.

A) Is an example of where our classifier says OBESE and the truth is NOT OBESE. This is an example of a False Positive.

B) Is an example of where our classifier says NOT OBESE and the truth is OBESE. This is an example of a False Negative.



Source: StatsQuest

Pros	**Cons**
Simple and easy to interpret	Sensitive to outliers
Computationally efficient	Not suitable for complex relationships
Outputs probabilities	Not effective with high-dimensional sparse data
Works well for linearly separable data	Assumes feature independence
Great of business decisions	
Based on solid statistical theory	



Logistic Regression Pros and Cons

Pros

1. Logistic regression is less prone to over-fitting than other classifiers.
2. It is computationally easy to train and predict new values.
3. Describes relationship between independent variables and dependent variables.

Cons

1. Main limitation of Logistic Regression is the assumption of linearity between the dependent variable and the independent variables.
2. If the number of observations are lesser than the number of features (ie; more columns than rows), Logistic Regression should not be used, otherwise it may lead to overfit.

Logistic Regression

```
# Imports
from pandas import DataFrame
from sklearn.linear_model import LogisticRegression

# All the Requisite Fake Data
training_data = DataFrame({ 'something': [1, 2, 3, 4], 'something_else': [5, 6, 7, 9],
                            'target':    [0, 0, 1, 1] })
X_to_predict =  DataFrame({ 'something': [1,    3],    'something_else': [5,       9] })

# Feature and Target Selection
X_train = training_data[['something', 'something_else']]
y_train = training_data['target']

# Training/Fitting
model = LogisticRegression()
model.fit(X_train, y_train)

# Predicting
y_pred = model.predict(X_to_predict)
print(y_pred)
```

```
[0 1]
```

Logistic Regression is Classification

1. Logistic regression is classification.
2. Logistic regression is classification.
3. Logistic regression is classification.

How to tell if our model is great or garbage.



Is my model great or garbage?

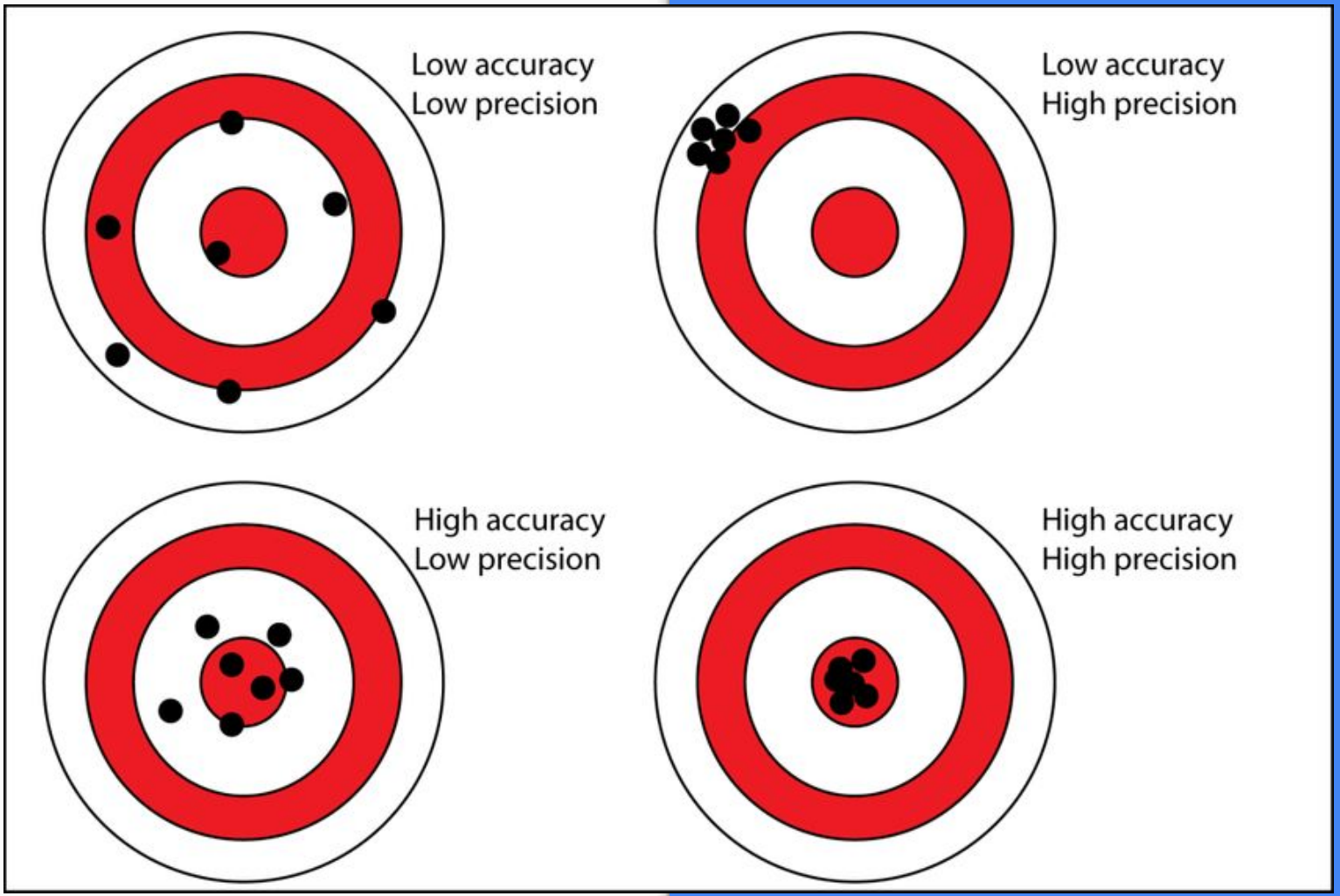
How you can tell with scoring metrics for your classification model



Evaluation Metrics for Classifiers (NOT REGRESSORS)

Metrics that tell you how well your model is performing.

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrices
- AUC score


Low accuracy
Low precision
Low accuracy
High precision
High accuracy
Low precision
High accuracy
High precision

ACCURACY

FEATURE | TARGET!! | y_pred

WEIGHT	IS OBESE	Model Prediction	Correct
0.4	NO	NO	✅
1.1	NO	NO	✅
2.1	NO	NO	✅
2.1	YES	NO	🚫
3.1	YES	YES	✅
5.5	YES	YES	✅

5 out of 6 correct

==

5 / 6 = 83% accuracy

Scoring Metrics for Classifiers

Metric	Bare-Bones Description
Accuracy	The percentage of predictions which are correct ((TP+TN) / (TP+TN+FP+FN))
Precision	The percentage of positive predictions which are correct (TP / (TP+FP))
Recall	The percentage of actual positives which are correct (TP / (TP+FN))
F1	The harmonic mean of precision and recall (2 × precision × recall / (precision+recall))
Metrics aren't limited to just those four. There's more…	

TRUE/FALSE == Model got it correct or wrong
Positive/Neg == did you actually have cancer or not

The Simple Confusion Matrix

	Predicted Negative	**Predicted Positive**
Actually Negative	True Negative	False Positive (Type 1 Error)
Actually Positive	False Negative (Type 2 Error)	True Positive



Shoutout to student Hanqi Lin for posting this image in slack

TRUE/FALSE == Model got it correct or wrong
Positive/Neg == did you actually have cancer or not

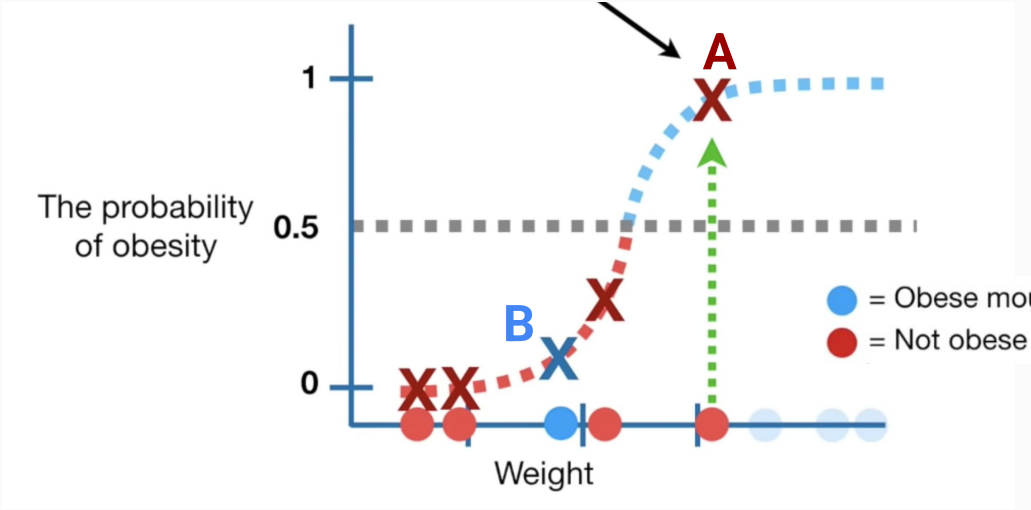
- **True Positive (TP):** The model correctly predicted a positive outcome i.e the actual outcome was positive.
- **True Negative (TN):** The model correctly predicted a negative outcome i.e the actual outcome was negative.
- **False Positive (FP):** The model incorrectly predicted a positive outcome i.e the actual outcome was negative. It is also known as a Type I error.
- **False Negative (FN):** The model incorrectly predicted a negative outcome i.e the actual outcome was positive. It is also known as a Type II error.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Classifiers can get things wrong.

A) Is an example of where our classifier says OBESE and the truth is NOT OBESE. This is an example of a False Positive.

B) Is an example of where our classifier says NOT OBESE and the truth is OBESE. This is an example of a False Negative.



Source: StatsQuest

Confusion Matrices *Can* Be Bigger

	Predicted 🐱	Predicted 🐶	Predicted 🐭
Actually 🐱	🐱 = 🐱 390	🐶 ≠ 🐱 3	🐭 ≠ 🐱 0
Actually 🐶	🐱 ≠ 🐶 2	🐶 = 🐶 93	🐭 ≠ 🐶 0
Actually 🐭	🐱 ≠ 🐭 0	🐶 ≠ 🐭 0	🐭 = 🐭 435

- Going down the matrix are the truths.
- Going across are the predictions.
- The diagonal cells (🟩) represent the correct predictions where the predicted class matches the actual class.
  - The others (🟥) are incorrect predictions.
- In each cell is a count of predictions.
- Summing all the cells in the matrix gives you the total count of predictions made.

Why we might want a different threshold

REDUCE TYPE 1 ERROR → INCREASE THRESHOLD

Reduce false positives == make it stricter in order for positive classification

Examples for increasing threshold

- Whether email is spam
- Deciding if someone is guilty or not

REDUCE TYPE 2 ERROR → DECREASE THRESHOLD

Reduce false negatives == less strict to catch more positive cases

Examples for decreasing threshold

- Whether someone has a disease, most medical exams/test (cancer, covid, etc)
- Deciding if a product is defective or not

```python
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

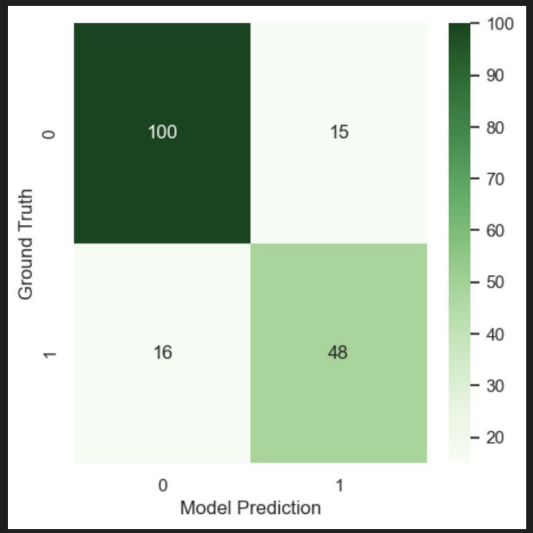
# ACCURACY
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score: %f" % accuracy)

# F1 ACCURACY
f1 = f1_score(y_test, y_pred)
print("F1 Score: %f" % f1)


# CONFUSION MATRIX
cm = confusion_matrix(y_test, y_pred)
fig = plt.figure(figsize=(5,5))
ax = sns.heatmap(cm, annot=True, cmap='Greens', fmt='g')
plt.ylabel('Ground Truth')
plt.xlabel('Model Prediction');
```

[38] ✓ 0.0s Python

```
Accuracy Score: 0.826816
F1 Score: 0.755906
```



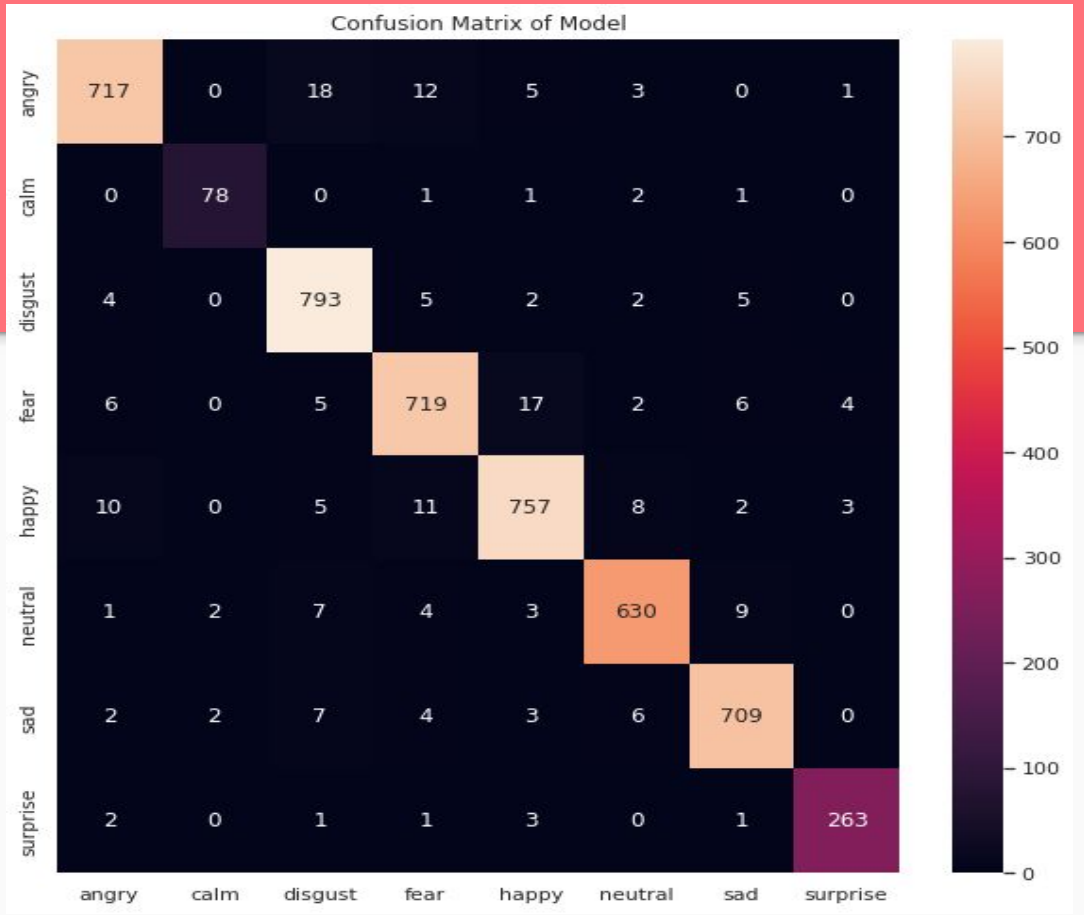
The metrics you need

Easy in sklearn

There are more we will do in later classes.

You can predict more than one class.

What the truth is



What the model predicted

Recall

TMI SKIP SLIDE

aka Sensitivity aka True Positive Rate

Not saying someone has cancer when they do have cancer is REALLY BAD. Whereas saying they do have cancer when they don't isn't as bad. This is when recall is important.

What proportion of actual positives was identified correctly?

Out of all the times THE ACTUAL was "YES", what percentage did you correctly label.

A perfect score is 1.0. A model that produces no false negatives has a recall of 1.0. You need high recall when the cost of not labeling YES is high (a false negative). For example, when detecting cancer.

$$\text{Recall} = TP / (TP + FN)$$

	NO (Predicted NO)	YES (Predicted YES)
NO (Truth is NO)	**True Negative** We said NO cancer Actual was NO cancer.	**False Positive (Type 1 Error)** We said YES cancer Actual was NO cancer
YES (Truth is YES)	**False Negative (Type 2 Error)** We said NO cancer Actual was YES cancer	**True Positive** We said YES cancer Actual was YES cancer

Precision

TMI SKIP SLIDE

What proportion of positive identifications was actually correct?

Out of all the times our model says “YES”, what percentage was it correct.

A perfect score is 1.0. A model that produces no false positives has a precision of 1.0.

When the models output is being used to act in an expensive manner, you need precision to be high. Sending a gift basket to a customer you think is going to churn is a great example.

Precision = TP / (TP + FP)

	NO (Predicted NO)	YES (Predicted YES)
NO (Truth is NO)	**True Negative** We said NO cancer Actual was NO cancer.	**False Positive** **(Type 1 Error)** We said YES cancer Actual was NO cancer
YES (Truth is YES)	**False Negative** **(Type 2 Error)** We said NO cancer Actual was YES cancer	**True Positive** We said YES cancer Actual was YES cancer

Be Very Careful

Understand the consequences of your model's scores.

	Predicted Cancer	Predicted No Cancer
Actually Cancer	1 TP	1 FN
Actually No Cancer	0 FP	98 TN

What does this mean?

Do you get peace of mind if this model says that you have NO CANCER? 🙂

THE MODEL SAYS YOU HAVE NO CANCER.

ACCURACY **99%**



PRECISION **100%**



RECALL **50%**



IF YOU ACTUALLY HAVE CANCER,
THE TEST IS AS GOOD AS A COIN TOSS.
ITS WRONG HALF THE TIME.

Be Very Careful

Take a fraud detection model for example.

- An **accurate** fraud detection model can let 100% of the fraud slip through. If 99.9% of transactions were not fraudulent, and the model *always* predicts no fraud, it would be 99.9% accurate.
- And a fraud detection model with high **recall** might catch 100% of the fraud—by simply flagging *every transaction* as fraud.
- If you optimize for *any metric alone*, be prepared to explain why. 🙂

Evaluation metrics recap.

Accuracy

- *Always between 0-1*
- *Higher the better 0.99 is great*

F1-Score

- *Always between 0-1*
- *Higher the better 0.99 is great*
- *Harmonic mean between precision and recall.*

Confusion Matrix

- *Visually inspect and sanity check your predictions vs the truth*



Always Be ISBEing



Inspect and explore the data. This is a very important part as all subsequent steps rely on a valid understanding of the data.



Select and engineer features. Don't include features which won't help predict the target, and don't forget to convert the data into the required format.



Build and train the model.



Evaluate the model using the metrics. You may need to go back to tweak the model parameters, select different features, or re-engineer.

Step by step guide

1. Select data
2. Clean data
3. Feature engineering
4. Define your X
5. Define your y
6. Split into train and test sets
7. Initialize model
8. Fit model with training data
9. Make predictions on test data
10. Evaluate your model with predictions vs testing.

1. Select
2. Clean
3. Define
4. Define
5. Split i
6. Initiali
7. Fit mo
8. Make
9. Evalua

```python
# Load data
df = pd.read_csv('data/titanic.csv')

# Clean data
df = df.dropna(subset=['age', 'fare'])

# Select your features and target variables
selected_features = ['fare', 'age']
target_feature = 'survived'

# Define your X and y
X = df[selected_features]
y = df[target_feature]

# Split your data into train and test.
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Initalize your model
model = LogisticRegression()

# Fit your model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate your model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy Score: %f" % accuracy)

f1 = f1_score(y_test, y_pred)
print("F1 Score: %f" % f1)
```

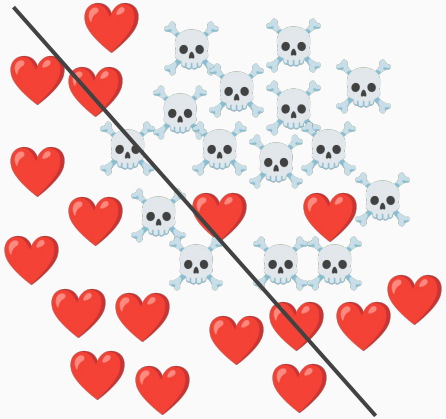
[11] ✓ 0.0s Python

```
Accuracy Score: 0.664804
```

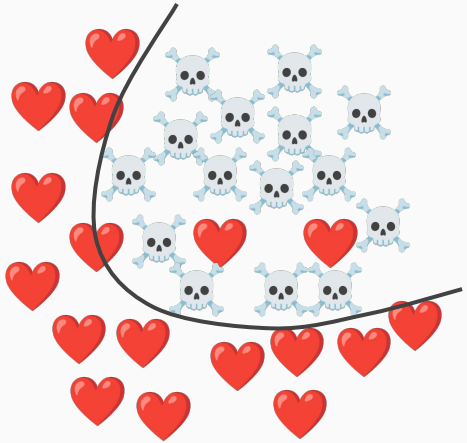
Jargon Alert

Over fitting
Coefficients
MLE
Log-Odds

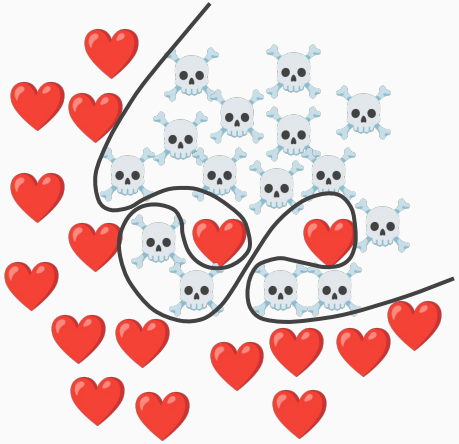
Underfitting and Overfitting



Underfitting
(too simple to explain variance)

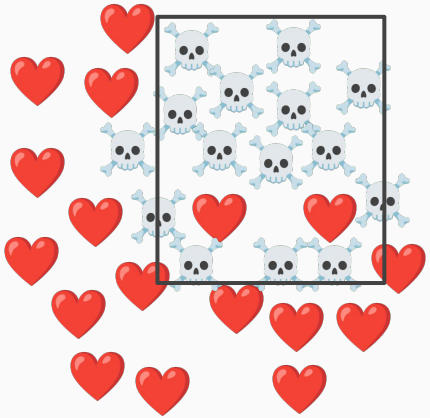


Appropriate Fitting

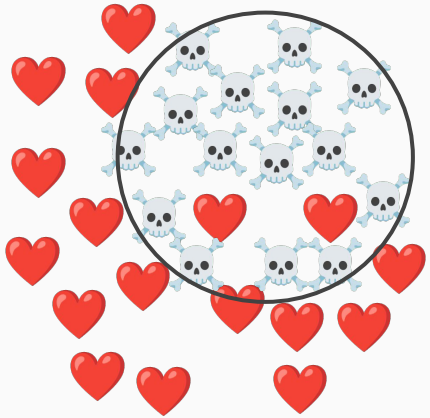


Overfitting
(force-fitting—too good to be true)

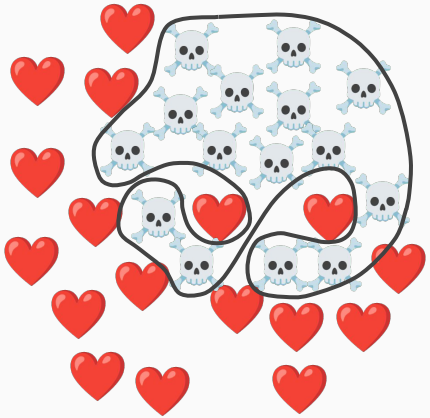
Underfitting and Overfitting



Underfitting
(too simple to explain variance)

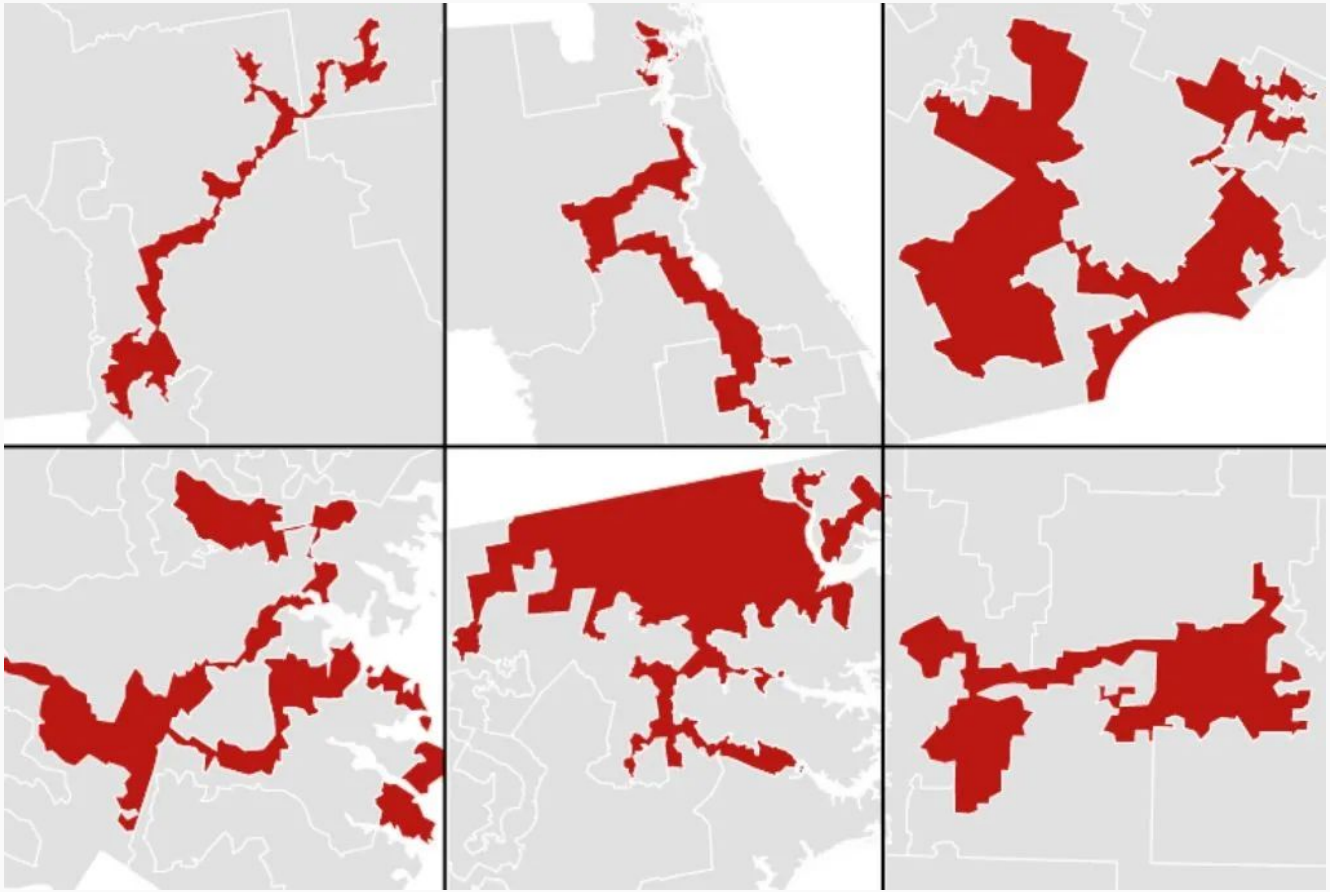


Appropriate Fitting



Overfitting
(force-fitting—too good to be true)

Is Gerrymandering Overfitting?





How to make money with logit.

~~Insights from your model.~~

So what?

I want to learn about my data, I want to be able to infer or make decisions based on our ML model, how does logistic regression do this?

Feature Importances:

How to interpret your results to make decisions and money using logistic regression.

Warning

Zack used ChatGPT to help generate a good example to illustrate interpreting features.

BEFORE USING ChatGPT

1. **You cannot blindly follow / trust what ChatGPT says.**
2. When it is wrong, it sounds very correct.
3. Don't become this guy: Lawyer Used ChatGPT In Court—And Cited Fake Cases. A Judge Is Considering Sanctions.
    a. (He even asked ChatGTP, are these cases real) and it said yes!
4. In the code it first generated, it made a big mistake, but the code worked and results looked good but the results were actually really bad.
5. If you use it to cheat in this class, you're just cheating yourself.

1. **You have to sanity check its results.**
2. To be able to do this, you have to know it is you are doing first.
3. Zack learned these feature important techniques from years of experience, teaching at Galvanize, and self learning.
4. He sanity checked them again by referring to these sources. [1] [2]

Logit | VS Code Jupyter Notebooks F

chat.openai.com/c/44136640-694c-4f8c-ac08-a737a026ef65

I'm trying to teach my college students how to explain the output of their stats model logistic regression results summary to their boss or marketing team. can you please provide an example in which the results show something like, this is the most important varible and increases the chance of someone buying something by 4x. please include a summary report like stats models does and everying in markdown format. thank you

Of course! I'll create a hypothetical logistic regression results summary, and then explain it as if you're presenting to a non-technical audience like a boss or marketing team.

Hypothetical Logistic Regression Results for Predicting Online Purchase:

markdown

Copy code

```
                           Logit Regression Results
==============================================================================
Dep. Variable:                 bought   No. Observations:
Model:                          Logit   Df Residuals:
```

Regenerate

Send a message

ChatGPT may produce inaccurate information about people, places, or facts. ChatGPT August 3 Version

```python
import statsmodels.api as sm
import pandas as pd

# Load the dataset we generated earlier
df = pd.read_csv("data/online_store_logit.csv")
df = df.drop(columns=['previous_customer'])
df.head()
```

✓ 0.0s

	user_id	gender	time_on_site_min	ads_seen	bought
0	1	Male	6.00	3	1
1	2	Female	3.60	6	0
2	3	Female	4.58	1	1
3	4	Male	0.14	1	0
4	5	Male	2.15	3	0

```python
# Prepare independent variables (X) and dependent variable (y)
X = pd.get_dummies(df[["gender", "time_on_site_min", "ads_seen"]], drop_first=True, dtype='int')
X = sm.add_constant(X)
y = df["bought"]
X.head()
```

✓ 0.0s

	const	time_on_site_min	ads_seen	gender_Male
0	1.0	6.00	3	1
1	1.0	3.60	6	0
2	1.0	4.58	1	0
3	1.0	0.14	1	1
4	1.0	2.15	3	1

```python
# Fit logistic regression model
model = sm.Logit(y, X).fit()

# Display the full summary report
print(model.summary())
```

✓ 0.0s

```
Optimization terminated successfully.
         Current function value: 0.591208
         Iterations 5
                           Logit Regression Results
==============================================================================
Dep. Variable:                 bought   No. Observations:                 1200
Model:                          Logit   Df Residuals:                     1196
Method:                           MLE   Df Model:                            3
Date:                Tue, 14 Oct 2025   Pseudo R-squ.:                  0.1164
Time:                        17:16:03   Log-Likelihood:                -709.45
converged:                       True   LL-Null:                       -802.94
Covariance Type:            nonrobust   LLR p-value:                 2.731e-40
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.2045      0.229     -9.620      0.000      -2.654      -1.755
time_on_site_min     0.4752      0.044     10.762      0.000       0.389       0.562
ads_seen          0.2991      0.040      7.519      0.000       0.221       0.377
gender_Male      -0.0180      0.128     -0.141      0.888      -0.269       0.233
==============================================================================
```

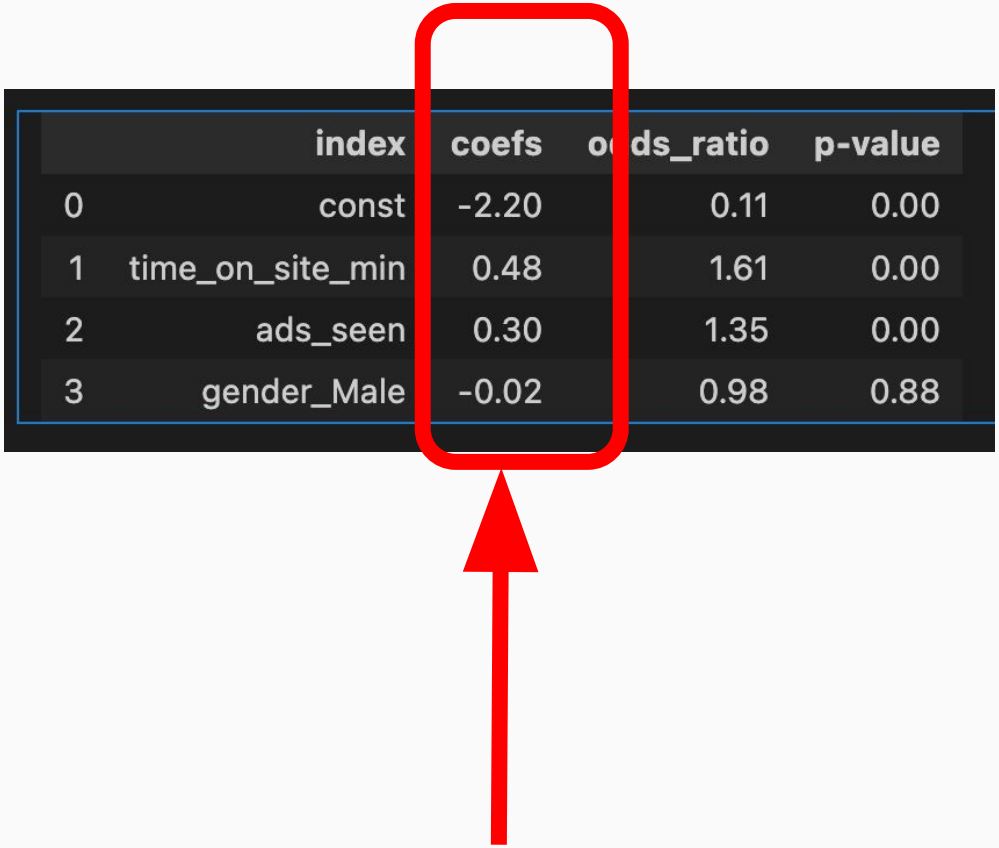
THE MOST IMPORTANT METRICS

```
1  d = dict()
2  d['coefs'] = model.params
3
4  param_df = pd.DataFrame.from_dict(d).reset_index()
5  param_df['odds_ratio'] = np.exp(param_df['coefs'])
6  param_df['p-value'] = [0.0,0.0,0.0,0.88]
7  param_df.round(2)
```

✓ 0.0s

	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88

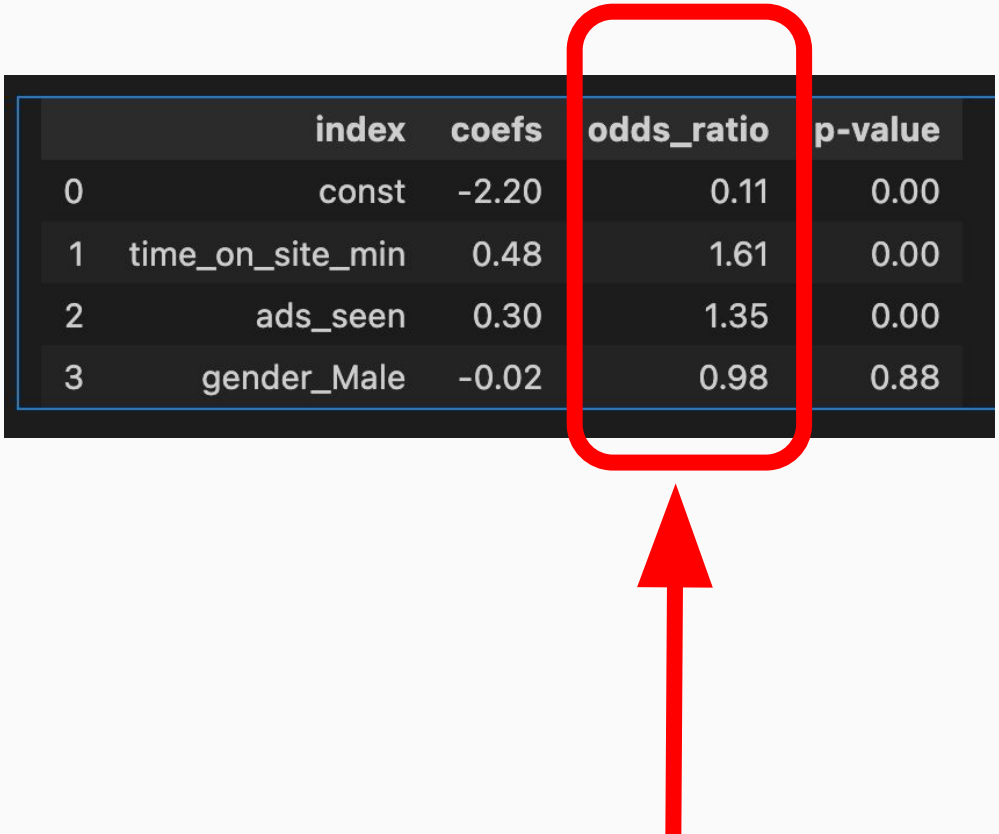
THE MOST IMPORTANT METRICS



	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88

`coef` → Coefficients:
:: Tell you the direction
more likely (+)
or less likely (-)
to buy

THE MOST IMPORTANT METRICS



	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88

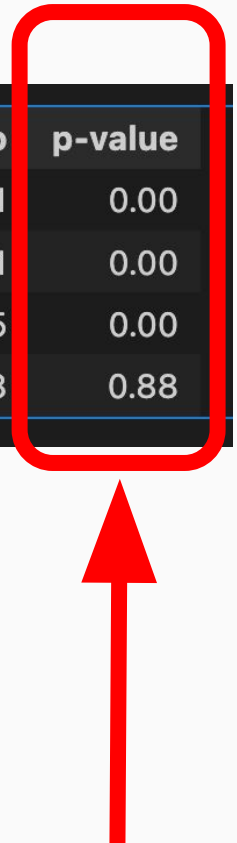
`Odds ratio`

Is just the exponential of the coefficient...

Makes it human readable/interpretable.

THE MOST IMPORTANT METRICS

	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88



P>|z| → P-Values:

* Tell you if the effect is significant
* Less than 0.05 is significant

THE MOST IMPORTANT METRICS

	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88

`Time on site`

For every 1 min increase in time...

The user is 1.6x more likely to buy

With statistical significant impact.

THE MOST IMPORTANT METRICS

	index	coefs	odds_ratio	p-value
0	const	-2.20	0.11	0.00
1	time_on_site_min	0.48	1.61	0.00
2	ads_seen	0.30	1.35	0.00
3	gender_Male	-0.02	0.98	0.88

Time on site

For every 1 min increase in time...

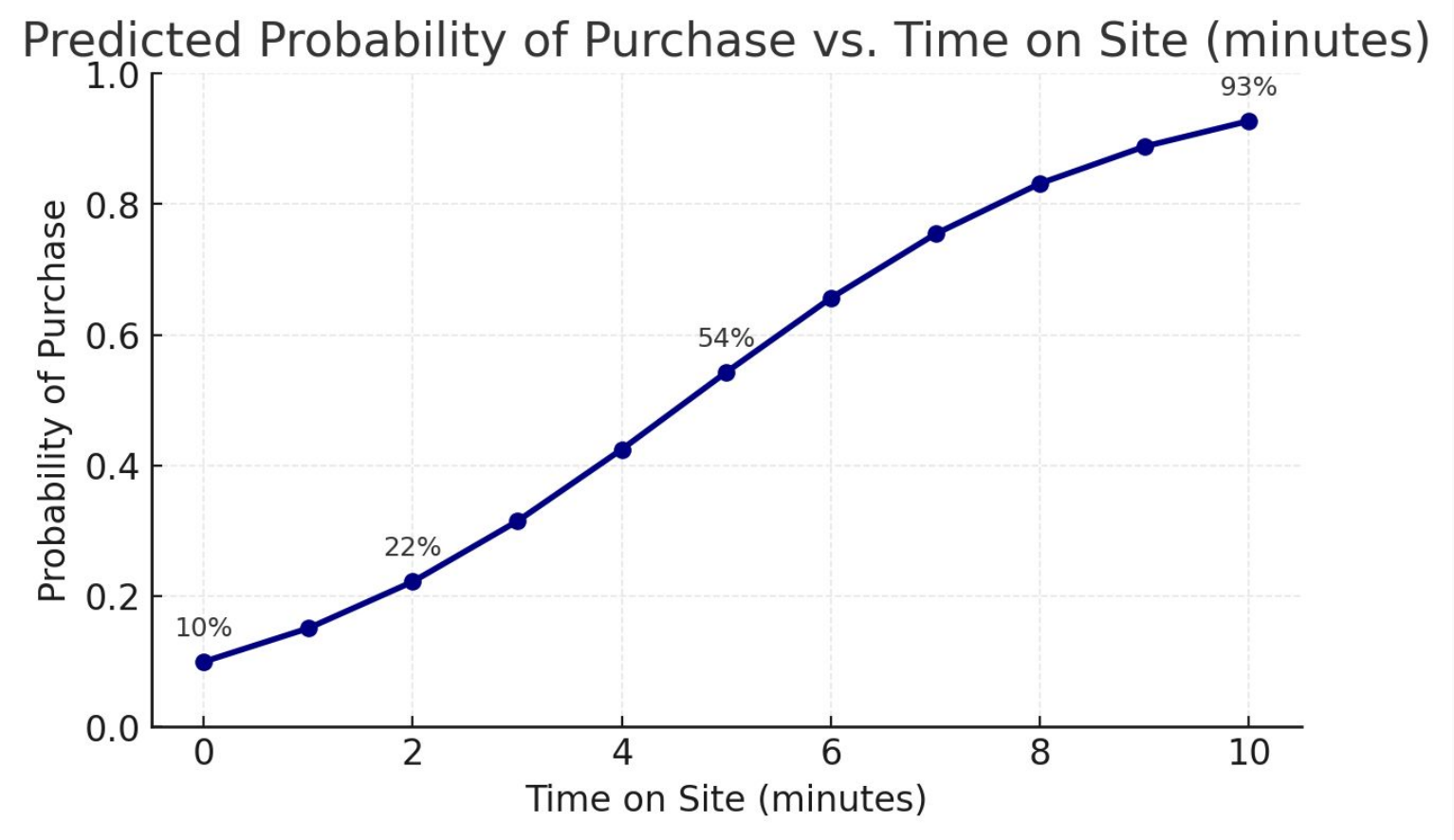
The user is 1.6x more likely

HOLDING ALL OTHER VARIABLES CONSTANT!

Only when you just these factors,
time_on_site
ads_seen,
And gender

Then you can say a 1 unit increase in time will increase odds by 1.61x or 61%

THE MOST IMPORTANT METRICS



It's all about the coefficients and p-values.

***Table was *ChatGPT generated*. And sanity checked by Zack.**

Logit Regression Results

Dep. Variable:	bought	No. Observations:	1000
Model:	Logit	Df Residuals:	993
Method:	MLE	Df Model:	6
Date:	Mon, 11 Sep 2023	Pseudo R-squ.:	0.4250
Time:	10:15:15	Log-Likelihood:	-402.20
converged:	True	LL-Null:	-700.50
Covariance Type:	nonrobust	LLR p-value:	2.00e-126

	coef	std err	z	P>|z|	[0.025	0.975]
const	-1.2550	0.124	-10.120	0.000	-1.498	-1.012
ad_viewed	1.3980	0.122	11.450	0.000	1.159	1.637
website_time	2.0835	0.116	17.980	0.000	1.856	2.311
age	0.0560	0.011	5.100	0.000	0.034	0.078
discount_code	-0.5050	0.111	-4.550	0.000	-0.723	-0.287
referral	-0.7050	0.125	-5.640	0.000	-0.950	-0.460
customer_review	0.0620	0.063	0.980	0.327	-0.062	0.186

THE MOST IMPORTANT METRICS

	coef	P>|z|
const	-1.2550	0.000
ad_viewed	1.3980	0.000
website_time	2.0835	0.000
age	0.0560	0.000
discount_code	-0.5050	0.000
referral	-0.7050	0.000
customer_review	0.0620	0.327

`coef` → Coefficients:

* Tell you HOW BIG THE EFFECT IS.
* and the direction (negative values decrease)

`P>|z|` → P-Values:

* Tell you if the effect is significant
* Less than 0.05 is significant

Coefficients

*Text was generated by ChatGTP and verified by Zack

	coef
const	-1.2550
ad_viewed	1.3980
website_time	2.0835
age	0.0560
discount_code	-0.5050
referral	-0.7050
customer_review	0.0620

Note: The provided explanations use the exponentiation of the coefficients (exp(coef)) to interpret the odds ratio, which gives a more intuitive understanding for non-technical audiences.

Ad Viewed:

* Importance: High
* Insight: Every advertisement a visitor views boosts the odds of them buying by about 4 times (exp(1.3980) = 4.05).

Our advertisements appear to be doing their job! → ***added by zack (HOLDING OTHER VARIABLES CONSTANT)***

Time Spent on Website (website_time):

* Importance: Very High
* Insight: Each extra minute a visitor dedicates to our website, the odds of them making a purchase increase by about 8 times (exp(2.0835) = 8.03). Engaging website content remains crucial.

Coefficients

*Text was assisted by ChatGTP and verified by Zack

	coef
const	-1.2550
ad_viewed	1.3980
website_time	2.0835
age	0.0560
discount_code	-0.5050
referral	-0.7050
customer_review	0.0620

age

* **Definition:** age in years
* **Coefficient:** 0.056
* **increase factor aka odds ratio:** 1.051 = exp(0.056)
* **percentage increase**: 5.1% == (1.051 - 1) * 100
* **p-value**: 0.00 significant.

Insight:

Every one unit increase in age increases the odds (likelihood) of them buying by 5.1%

Y_pred = bought something 0,1

Age = [100, 25, 22]

Coefficients

*Text was assisted by ChatGTP and verified by Zack

	coef
const	-1.2550
ad_viewed	1.3980
website_time	2.0835
age	0.0560
discount_code	-0.5050
referral	-0.7050
customer_review	0.0620

age

* **Definition:** age in years
* **Coefficient:** 0.056
* **increase factor:** 1.051 = exp(0.056)
* **percentage increase**: 5.1% == (1.051 - 1) * 100
* **p-value**: 0.00 significant.

Insight:

Every one unit increase in age increases the odds of them buying by 5.1%

Coefficients

*Text was assisted by ChatGTP and verified by Zack

	coef
const	−1.2550
ad_viewed	1.3980
website_time	2.0835
age	0.0560
discount_code	−0.5050
referral	−0.7050
customer_review	0.0620

HOLDING ALL OTHER VARIABLES CONSTANT!

Only when you just these factors,
ad_views
website_time,
age, discount_code ...

Then you can say Age will increase odds by 5.1%

Insight:

Every one unit increase in age increases the odds of them buying by 5.1%

5. For Age 100:

$$\text{Odds at } 100 = 1.051^{100} \approx 141.957$$

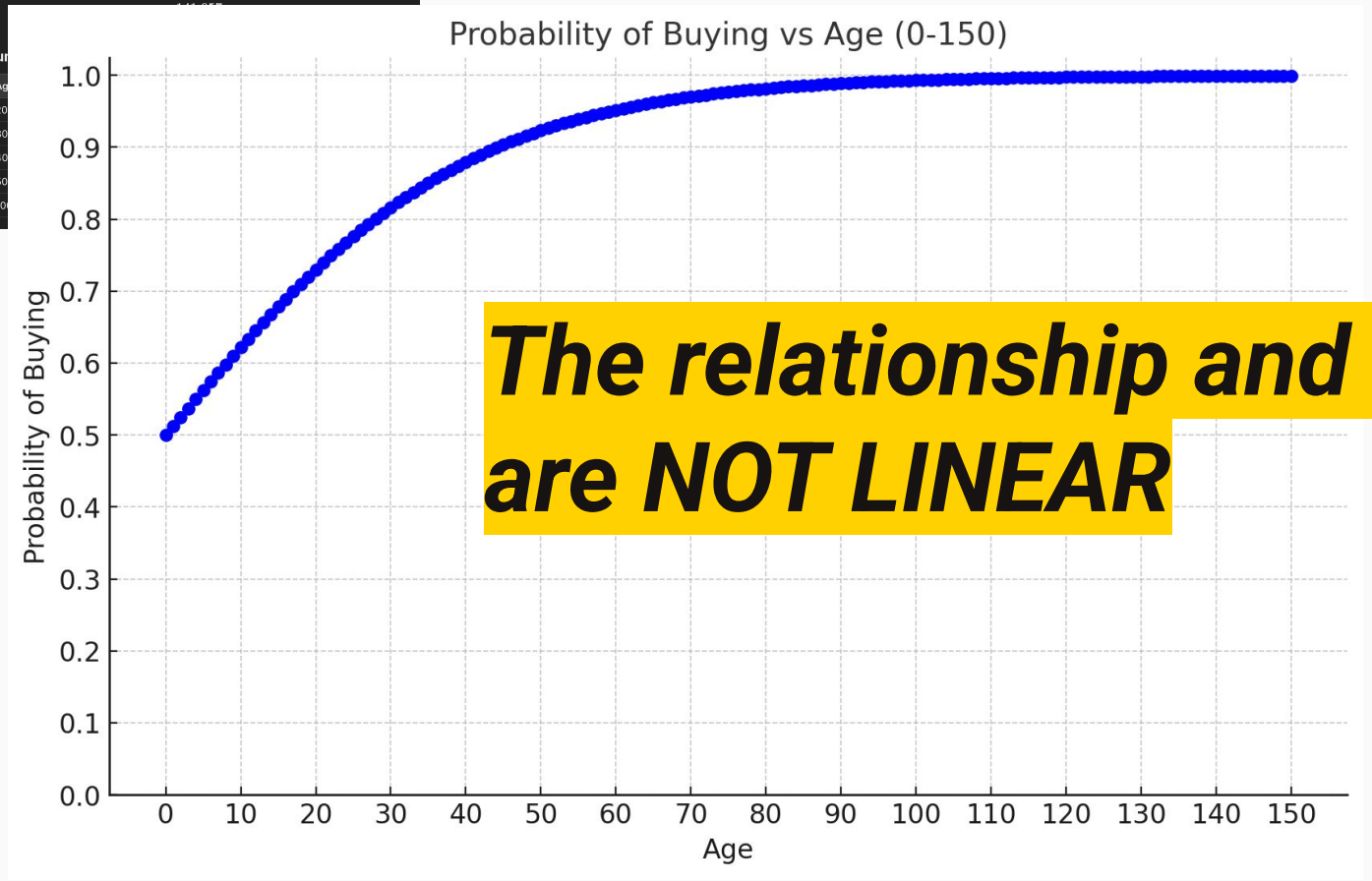
$$\text{Probability at } 100 = \frac{141.957}{1 + 141.957} \approx 0.993 \quad (\text{or } 99.3\%)$$

Summary of Odds and Probabilities for Different Ages

Age	Odds of Buying	Probability of Buying
20	2.712	73.1%
30	4.441	81.6%
40	7.270	87.9%
50	11.905	92.2%
100	141.957	99.3%

5. For Age 100:

Odds at $100 = 1.051^{100} \approx 141.957$



The relationship and effects are NOT LINEAR

Coefficients

*Text was assisted by ChatGTP and verified by Zack

	coef
const	-1.2550
ad_viewed	1.3980
website_time	2.0835
age	0.0560
discount_code	-0.5050
referral	-0.7050
customer_review	0.0620

website_time

* **Definition:** Time Spent on Website:
* **Coefficient:** 2.083
* **increase factor** aka **odds ratio:** 8.03 = exp(2.083)
* **p-value**: 0.00 significant.

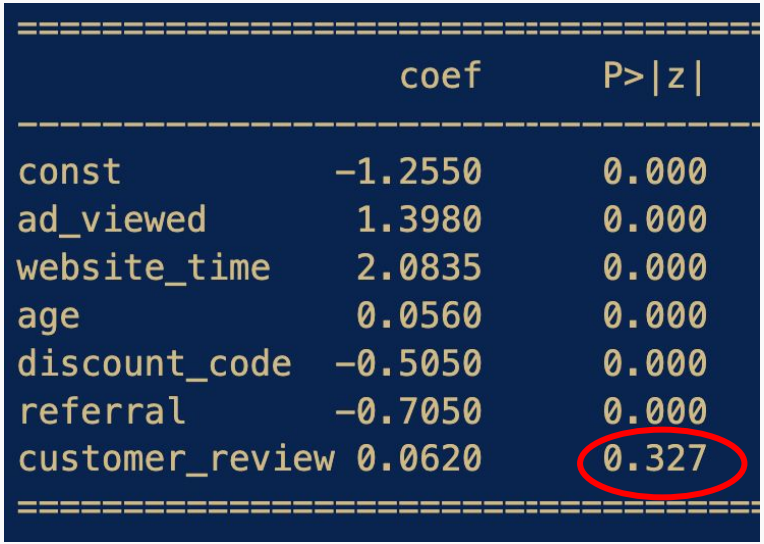
Insight:

Each extra minute a visitor is on the site, the odds of them making a purchase increase by a factor of ~8x

Note: The provided explanations use the exponentiation of the coefficients (exp(coef)) to interpret the odds ratio, which gives a more intuitive understanding for non-technical audiences.

How to interpret / understand coefficients and p-values.

***Text was generated by ChatGTP and verified by Zack**



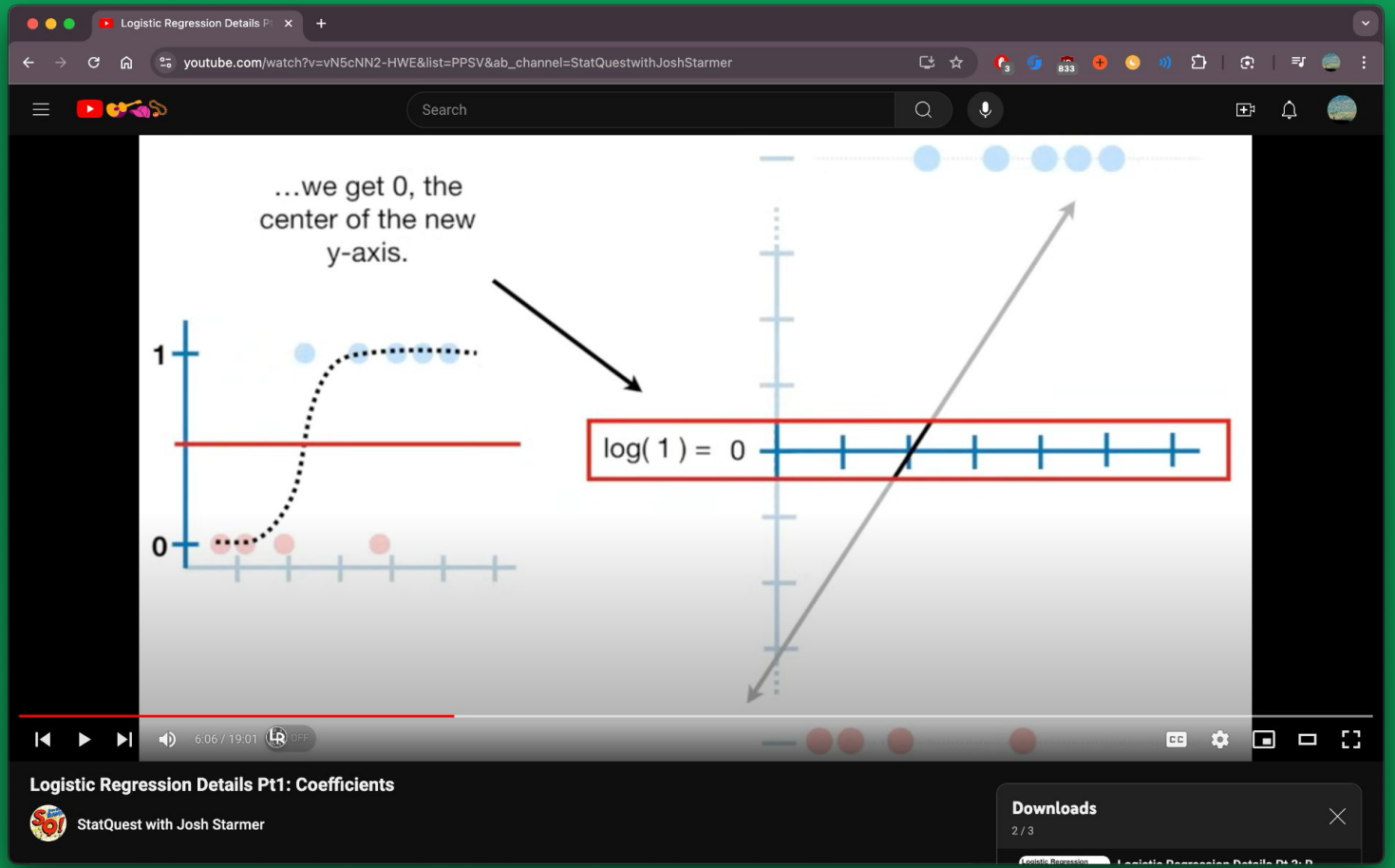
	coef	P>|z|
const	-1.2550	0.000
ad_viewed	1.3980	0.000
website_time	2.0835	0.000
age	0.0560	0.000
discount_code	-0.5050	0.000
referral	-0.7050	0.000
customer_review	0.0620	0.327

Customer Reviews:

*** Importance: Uncertain - low p-value**

* Insight: Even though one might assume that reading more customer reviews would influence purchasing decisions, our analysis shows that this isn't a statistically significant predictor. **The p-value is above our typical threshold (0.05), indicating we can't be confident in its influence.** It might be worth reevaluating this at a later stage with more data.

Note: The provided explanations use the exponentiation of the coefficients (exp(coef)) to interpret the odds ratio, which gives a more intuitive understanding for non-technical audiences.


Logistic Regression Details Pt
youtube.com/watch?v=vN5cNN2-HWE&list=PPSV&ab_channel=StatQuestwithJoshStarmer
Search
...we get 0, the center of the new y-axis.
1
0
log(1) = 0
6:06 / 19:01
Logistic Regression Details Pt1: Coefficients
StatQuest with Josh Starmer
Downloads
2 / 3

(generated by GPT)

Main Takeaway :

- Ads and website engagement remain key!
- However, our promotions and referrals need some fine-tuning.
- And, while customer reviews are valuable for many reasons, as of now, they don't appear to be a decisive factor in purchase decisions based on this data.

TERMS TO PUT ON YOUR RADAR

- *Training and testing.*
- *Scaling Data*
- *Sigmoid and MLE*
- *Gradient Descent*
- *Hyper-parameters*
- *Ridge and Lasso*
- *Iterations*
- *Balanced / Unbalanced Data*
  - *There should be equal examples of surived / died in your data.*
- *Feature Independence // MULTICOLLINEARITY*
  - *Ideally,,, the features / columns you are using to predict should NOT BE RELATED AT ALL and no correlation between them.*
- *Accuracy Fallacy*


But… Sorry,,,
not enough time